


```
In [*]: import torch
        from torchvision import datasets, transforms

        data_train = datasets.MNIST(root = "data/", transform = transform, train = True, download = True)

        data_test = datasets.MNIST(root = "data/", transform = transform, train = False )

Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to data/MNIST\raw\train-images-idx3-ubyte.gz
9920512it [00:27, 359076.98it/s]

Extracting data/MNIST\raw\train-images-idx3-ubyte.gz to data/MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz to data/MNIST\raw\train-labels-idx1-ubyte.gz
32768it [00:01, 20074.99it/s]

Extracting data/MNIST\raw\train-labels-idx1-ubyte.gz to data/MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz to data/MNIST\raw\t10k-images-idx3-ubyte.gz
0it [00:00, ?it/s]
```

```
In [3]: import torch
        from torchvision import datasets, transforms

        data_train = datasets.MNIST(root = "data/", transform = transform, train = True, download = True)

        data_test = datasets.MNIST(root = "data/", transform = transform, train = False )

Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to data/MNIST\raw\train-images-idx3-ubyte.gz
9920512it [00:27, 359076.98it/s]

Extracting data/MNIST\raw\train-images-idx3-ubyte.gz to data/MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz to data/MNIST\raw\train-labels-idx1-ubyte.gz
32768it [00:01, 20074.99it/s]

Extracting data/MNIST\raw\train-labels-idx1-ubyte.gz to data/MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz to data/MNIST\raw\t10k-images-idx3-ubyte.gz
1654784it [00:06, 254144.32it/s]

Extracting data/MNIST\raw\t10k-images-idx3-ubyte.gz to data/MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz to data/MNIST\raw\t10k-labels-idx1-ubyte.gz
8192it [00:00, 12992.98it/s]

Extracting data/MNIST\raw\t10k-labels-idx1-ubyte.gz to data/MNIST\raw
Processing...
Done!
```

其中，root用于指定数据集在下载之后的存放路径，这里存放在根目录下的data文件夹中；

transform用于指定导入数据集时需要对数据进行哪种变换操作，在后面会介绍详细的变换操作类型，**注意，要提前定义这些变换操作**；

train用于指定在数据集下载完成后需要载入哪部分数据，如果设置为True，则说明载入的是该数据集的训练集部分；如果设置为False，则说明载入的是该数据集的测试集部分。

4.2 PyTorch之torch.transforms

在torch.transforms中提供了丰富的类对载入的数据进行变换。

在计算机视觉中处理的数据集有很大一部分是图片类型的，而在PyTorch中实际进行计算的是Tensor数据类型的变量，所以我们首先需要解决的是数据类型转换的问题，如果获取的数据是格式或者大小不一的图片，则还需要进行归一化和大小缩放等操作,这些方法在torch.transforms中都能找到。

在torch.transforms中有大量的数据变换类，其中有很大一部分可以用于实现**数据增强（Data Argumentation）**。

若在我们需要解决的问题上能够参与到模型训练中的图片数据非常有限，则这时就要通过对有限的图片数据进行各种变换，来生成新的训练集了，这些变换可以是**缩小或者放大图片的大小、对图片进行水平或者垂直翻转**等，都是数据增强的方法。

在手写数字识别的问题上可以不使用数据增强的方法，因为可用于模型训练的数据已经足够了。对数据进行载入及有相应变化的代码如下：

```
In [4]: transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Lambda(lambda x: x.repeat(3,1,1)), ##图片格式是灰度图只有一个channel，将其变成RGB的3个channel
            transforms.Normalize(mean = [0.5,0.5,0.5], std = [0.5,0.5,0.5])
        ]
    )
```

代码中的torchvision.transforms.Compose类看作一种容器，它能够同时对多种数据变换进行组合。传入的参数是一个列表，列表中的元素就是对载入的数据进行的各种变换操作。

在以上代码中，在 `torchvision.transforms.Compose` 中只使用了一个类型的转换变换 `transforms.ToTensor` 和一个数据标准化变换 `transforms.Normalize`。

这里使用的标准化变换也叫作标准差变换法，这种方法需要使用原始数据的均值（Mean）和标准差（Standard Deviation）来进行数据的标准化，在经过标准化变换之后，数据全部符合均值为0、标准差为1的标准正态分布。

不过我们在这里偷了一个懒，均值和标准差的值并非来自原始数据的，而是自行定义了一个，不过仍然能够达到我们的目的。

下面介绍几个在 `torchvision.transforms` 中常用的数据变换操作。

(1) **`torchvision.transforms.Resize`**：用于对载入的图片数据按我们需求的大小进行缩放。传递给这个类的参数可以是一个整型数据，也可以是一个类似于 `(h,w)` 的序列，其中，`h`代表高度，`w`代表宽度，但是如果使用的是一个整型数据，那么表示缩放的宽度和高度都是这个整型数据的值。

(2) **`torchvision.transforms.Scale`**：用于对载入的图片数据按我们需求的大小进行缩放，用法和 `torchvision.transforms.Resize` 类似。

(3) **`torchvision.transforms.CenterCrop`**：用于对载入的图片以图片中心为参考点，按我们需要的大小进行裁剪。传递给这个类的参数可以是一个整型数据，也可以是一个类似于 `(h,w)` 的序列。

(4) **`torchvision.transforms.RandomCrop`**：用于对载入的图片按我们需要的大小进行随机裁剪。传递给这个类的参数可以是一个整型数据，也可以是一个类似于 `(h,w)` 的序列。

(5) **`torchvision.transforms.RandomHorizontalFlip`**：用于对载入的图片按随机概率进行水平翻转。我们可以通过传递给这个类的参数自定义随机概率，如果没有定义，则使用默认的概率值0.5。

(6) **`torchvision.transforms.RandomVerticalFlip`**：用于对载入的图片按随机概率进行垂直翻转。我们可以通过传递给这个类的参数自定义随机概率，如果没有定义，则使用默认的概率值0.5。

(7) **`torchvision.transforms.ToTensor`**：用于对载入的图片数据进行类型转换，将之前构成PIL图片的数据转换成Tensor数据类型的变量，让PyTorch能够对其进行计算和处理。

(8) **`torchvision.transforms.ToPILImage`**：用于将Tensor变量的数据转换成PIL图片数据，主要是为了方便图片内容的显示。

4.3 数据预览和数据装载

在数据下载完成并且载入后，我们还需要对数据进行装载。

我们可以将数据的载入理解为对图片的处理，在处理完成后，我们就需要将这些图片打包好送给我们的模型进行训练了，而装载就是这个打包的过程。

在装载时通过 `batch_size` 的值来确认每个包的大小，通过 `shuffle` 的值来确认是否在装载的过程中打乱图片的顺序。装载图片的代码如下：

```
In [5]: data_loader_train = torch.utils.data.DataLoader(dataset = data_train,
                                                    shuffle=True,
                                                    batch_size=64)
data_loader_test = torch.utils.data.DataLoader(dataset = data_test,
                                                shuffle=True,
                                                batch_size=64)
```

对数据的装载使用的是 `torch.utils.data.DataLoader` 类，类中的 `dataset` 参数用于指定我们载入的数据集名称，`batch_size` 参数设置了每个包中的图片数据个数，代码中的值是64，所以在每个包中会包含64张图片。

将 `shuffle` 参数设置为 `True`，在装载的过程会将数据随机打乱顺序并进行打包。

在装载完成后，我们可以选取其中一个批次的数据进行预览。进行数据预览的代码如下：

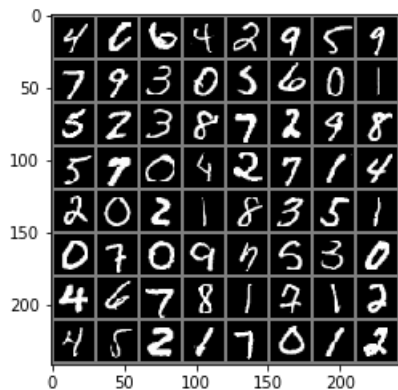
```
In [6]: import torchvision
import matplotlib.pyplot as plt
%matplotlib inline

images, labels = next(iter(data_loader_train))
img = torchvision.utils.make_grid(images)

img = img.numpy().transpose(1, 2, 0)
std = [0.5, 0.5, 0.5]
mean = [0.5, 0.5, 0.5]
img = img*std + mean
print([labels[i] for i in range(64)])
plt.imshow(img)
```

```
[tensor(4), tensor(0), tensor(6), tensor(4), tensor(2), tensor(9), tensor(5), tensor(9), tensor(7), tensor(9),
tensor(3), tensor(0), tensor(5), tensor(6), tensor(0), tensor(1), tensor(5), tensor(2), tensor(3), tensor(8),
tensor(7), tensor(2), tensor(9), tensor(8), tensor(5), tensor(7), tensor(0), tensor(4), tensor(2), tensor(7),
tensor(1), tensor(4), tensor(2), tensor(0), tensor(2), tensor(1), tensor(8), tensor(3), tensor(5), tensor(1),
tensor(0), tensor(7), tensor(0), tensor(9), tensor(7), tensor(5), tensor(3), tensor(0), tensor(4), tensor(6),
tensor(7), tensor(8), tensor(1), tensor(7), tensor(1), tensor(2), tensor(4), tensor(5), tensor(2), tensor(1),
tensor(7), tensor(0), tensor(1), tensor(2)]
```

```
Out[6]: <matplotlib.image.AxesImage at 0x1ec49e739b0>
```



在以上代码中使用了`iter`和`next`来获取一个批次的图片数据和其对应的图片标签，然后使用`torchvision.utils`中的`make_grid`类方法将一个批次的图片构造成网格模式。需要传递给`torchvision.utils.make_grid`的参数就是一个批次的装载数据，每个批次的装载数据都是4维的，维度的构成从前往后分别为`batch_size`、`channel`、`height`和`weight`，分别对应一个批次中的数据个数、每张图片的色彩通道数、每张图片的高度和宽度。在通过`torchvision.utils.make_grid`之后，图片的维度变成了

`(channel,height,width)`，这个批次的图片全部被整合到了一起，所以在这个维度中对应的值也和之前不一样了，但是色彩通道数保持不变。

我们想使用Matplotlib将数据显示成正常的图片形式，则使用的数据首先必须是数组，其次这个数组的维度必须是

`(height,width,channel)`，即色彩通道数在最后面。所以我们要通过`numpy`和`transpose`完成原始数据类型的转换和数据维度的交换，这样才能够使用Matplotlib绘制出正确的图像。在完成数据预览的代码中，我们先打印输出了这个批次中的全部标签，然后才对这个批次中的所有图片数据进行显示。

全连接网络实战MNIST手写字识别

```
In [7]: # 1 加载必要的库
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
```

```
In [8]: # 2 定义超参数
BATCH_SIZE = 16 # 每批处理的数据个数
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu") # 是使用GPU还是CPU进行训练
EPOCHS = 10 # 训练数据集的轮数
```

```
In [9]: # 3 构建pipeline, 对图像进行处理
pipeline = transforms.Compose([
    transforms.ToTensor(), #将图片转换成tensor
    transforms.Normalize((0.1307,), (0.3081,)) # 正则化, 用来降低模型复杂度
])
```

```
In [10]: # 4 下载、加载数据
from torch.utils.data import DataLoader

# 下载数据集
data_train = datasets.MNIST(root="data/", transform=pipeline, train=True, download=True)

data_test = datasets.MNIST(root="data/", transform=pipeline, train=False, download=True)

# 加载数据
train_loader = DataLoader(data_train, batch_size=BATCH_SIZE, shuffle=True)

test_loader = DataLoader(data_test, batch_size=BATCH_SIZE, shuffle=True)
```

```
In [11]: # 5 构建网络模型
class FCNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(FCNet, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out
```

```
In [12]: # 6 定义优化器

## 补充定义超参数
INPUT_SIZE = 784
HIDDEN_SIZE = 512
OUTPUT_SIZE = 10
LEARNING_RATE = 0.01

model = FCNet(INPUT_SIZE, HIDDEN_SIZE, OUTPUT_SIZE).to(DEVICE)

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

```
In [13]: # 7 定义训练方法
total_step = len(train_loader)
def train_model(model, device, train_loader, optimizer, epoch):
    # 模型训练
    model.train()
    for batch_index, (data, label) in enumerate(train_loader):
        # 部署到DEVICE上
        data, label = data.reshape(-1, 28 * 28).to(device), label.to(device)
        # 将模型参数的梯度初始化为0
        optimizer.zero_grad()
        # 前向传播计算预测值
        output = model(data)
        # 计算当前损失
        loss = F.cross_entropy(output, label)
        # 反向传播计算梯度
        loss.backward()
        # 更新所有参数
        optimizer.step()
        if batch_index % 3000 == 0:
            print('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'.format(epoch, EPOCHS, batch_index+1, total_step, lo
```

```
In [14]: # 8 定义测试方法
def test_model(model, device, test_loader):
    # 模型验证
    model.eval()
    # 正确率
    correct = 0.0
    # 测试损失
    test_loss = 0.0
    with torch.no_grad(): # 在测试阶段，不用计算梯度，也不用进行反向传播
        for data, label in test_loader:
            # 部署到DEVICE上
            data, label = data.reshape(-1, 28 * 28).to(device), label.to(device)
            # 测试数据
            output = model(data)
            # 计算测试损失
            test_loss += F.cross_entropy(output, label).item()
            # 找到概率最大值的下标
            _, predict = torch.max(output, dim=1) # 这里返回最大值和最大值索引
            # 累计正确的值
            correct += predict.eq(label.view_as(predict)).sum().item()
    test_loss /= len(test_loader.dataset)
    print("Test -- Average loss : {:.4f}, Accuracy : {:.3f}\n".format(test_loss, 100.0 * correct / len(test_l
```

```
In [15]: # 9 调用方法 7 / 8
for epoch in range(1, EPOCHS + 1):
    train_model(model, DEVICE, train_loader, optimizer, epoch)
    test_model(model, DEVICE, test_loader)
```

```
Epoch [1/10], Step [1/3750], Loss: 2.2341
Epoch [1/10], Step [3001/3750], Loss: 0.2403
Test -- Average loss : 0.0201, Accuracy : 91.840
```

```
Epoch [2/10], Step [1/3750], Loss: 0.3156
Epoch [2/10], Step [3001/3750], Loss: 0.1428
Test -- Average loss : 0.0182, Accuracy : 93.600
```

```
Epoch [3/10], Step [1/3750], Loss: 1.4725
Epoch [3/10], Step [3001/3750], Loss: 0.5390
Test -- Average loss : 0.0212, Accuracy : 92.610
```

```
Epoch [4/10], Step [1/3750], Loss: 0.3503
Epoch [4/10], Step [3001/3750], Loss: 0.1021
Test -- Average loss : 0.0197, Accuracy : 92.900
```

```
Epoch [5/10], Step [1/3750], Loss: 0.0204
Epoch [5/10], Step [3001/3750], Loss: 0.1797
Test -- Average loss : 0.0260, Accuracy : 91.010
```

运行结果如下：

Epoch [1/10], Step [1/3750], Loss: 2.2341 Epoch [1/10], Step [3001/3750], Loss: 0.2403 Test -- Average loss : 0.0201, Accuracy : 91.840

Epoch [2/10], Step [1/3750], Loss: 0.3156 Epoch [2/10], Step [3001/3750], Loss: 0.1428 Test -- Average loss : 0.0182, Accuracy : 93.600

Epoch [3/10], Step [1/3750], Loss: 1.4725 Epoch [3/10], Step [3001/3750], Loss: 0.5390 Test -- Average loss : 0.0212, Accuracy : 92.610

Epoch [4/10], Step [1/3750], Loss: 0.3503 Epoch [4/10], Step [3001/3750], Loss: 0.1021 Test -- Average loss : 0.0197, Accuracy : 92.900

Epoch [5/10], Step [1/3750], Loss: 0.0204 Epoch [5/10], Step [3001/3750], Loss: 0.1797 Test -- Average loss : 0.0260, Accuracy : 91.010

Epoch [6/10], Step [1/3750], Loss: 0.8168 Epoch [6/10], Step [3001/3750], Loss: 0.0041 Test -- Average loss : 0.0216, Accuracy : 93.280

Epoch [7/10], Step [1/3750], Loss: 0.1741 Epoch [7/10], Step [3001/3750], Loss: 0.0133 Test -- Average loss : 0.0193, Accuracy : 92.810

Epoch [8/10], Step [1/3750], Loss: 0.1954 Epoch [8/10], Step [3001/3750], Loss: 0.1188 Test -- Average loss : 0.0234, Accuracy : 93.140

Epoch [9/10], Step [1/3750], Loss: 0.1249 Epoch [9/10], Step [3001/3750], Loss: 0.7275 Test -- Average loss : 0.0234, Accuracy : 92.080

Epoch [10/10], Step [1/3750], Loss: 0.0704 Epoch [10/10], Step [3001/3750], Loss: 0.3561 Test -- Average loss : 0.0277, Accuracy : 92.500

从结果可以看出全连接网络训练的结果还有很大提升的空间。

为了验证我们训练的模型是不是真的如结果显示的一样的效果，我们随机选取一部分测试集中的图片，用训练好的模型进行预测，并对结果进行可视化。

代码如下：

```
In [35]: sample_test = torch.utils.data.DataLoader(dataset = data_test,
                                                    shuffle=True,
                                                    batch_size=4)

X_test, Y_test = next(iter(sample_test))
inputs = X_test.reshape(-1, 28 * 28).to(DEVICE)
pred = model(inputs)
_, pred = torch.max(pred, 1)

print("Predict label is :", [i for i in pred.data])
print("Real label is :", [i for i in Y_test])

img = torchvision.utils.make_grid(X_test)
img = img.numpy().transpose(1, 2, 0)

# std = [0.1307, 0.1307, 0.1307]
# mean = [0.3081, 0.3081, 0.3081]
std = (0.1307,)
mean = (0.3081,)
img = img*std+mean
plt.imshow(img)
```

Predict label is : [tensor(7, device='cuda:0'), tensor(9, device='cuda:0'), tensor(1, device='cuda:0'), tensor(5, device='cuda:0')]

Real label is : [tensor(7), tensor(9), tensor(1), tensor(5)]

Out [35]: <matplotlib.image.AxesImage at 0x1ec1720c630>

